

Chapitre n° 4

RECHERCHE DICHOTOMIQUE DANS UN TABLEAU TRIÉ

I RECHERCHE DICHOTOMIQUE DANS UN TABLEAU TRIÉ

1 PRINCIPE ET ALGORITHME

Le coût d'une recherche linéaire (Chapitre 3) d'un élément dans un tableau de taille n est linéaire en n dans le pire des cas qui consiste à tester tous les éléments du tableau. Le cas d'un tableau dont les éléments sont triés, par exemple si les éléments sont rangés dans l'ordre croissant, n'améliore pas cette complexité.

L'intérêt de la méthode dichotomique, est d'accélérer grandement la recherche. Mais cette méthode n'est applicable qu'à une condition : que le tableau soit trié. Le principe est de comparer l'élément avec celui se trouvant au milieu du tableau. S'ils sont égaux, c'est terminé, sinon on recommence avec la partie du tableau pouvant contenir l'élément.

Voici une application de cette méthode sur un exemple.

On recherche 9 dans $t = [1, 3, 5, 6, 9, 12, 14]$

On cherche dans $t[0:7]$	1	3	5	6	9	12	14
On compare $x=9$ et $t[3]=6$	1	3	5	6	9	12	14
On cherche dans $t[4:7]$ $t[3]=6$					9	12	14
On compare $x=9$ et $t[5]=12$					9	12	14
On cherche dans $t[4:4]$					9		
On compare $x=9$ et $t[4]=9$					9		

Voici une version possible :

```

1 def dichotomique(x, liste):
2     g, d = 0, len(liste)
3     while g < d - 1:
4         m = (g + d) // 2
5         if x < liste[m]:
6             d = m
7         else:
8             g = m
9     if liste[g] == x:
10         return g
11     return (None)

```

2 TERMINAISON

Ici on prouve que l'algorithme se termine.

Nous utilisons l'expression $d - g$ qui à chaque étape représente la taille de la sous-liste qui peut contenir x . Cette expression est appelée un **variant de boucle**.

Soit p le plus grand entier tel que la taille n du tableau est inférieure à 2^p . Avant d'entrer dans la boucle $d - g \leq 2^p$.

Après k itérations, $d - g \leq \frac{2^p}{2^k}$, soit $d - g \leq 2^{p-k}$. La suite des valeurs de l'expression $d - g$ est strictement décroissante et après p étapes, $d - g \leq 1$, donc $g \geq d - 1$ et la boucle est terminée. Par exemple, il faut sept étapes pour une taille de tableau égale à 100 et 10 étapes pour une taille égale à 1000.

Ceci prouve aussi que le coût de cette recherche dichotomique est de l'ordre du nombre de chiffres dans l'écriture binaire de n , donc de l'ordre de $\log_2(n)$. Ce coût est nettement inférieur à celui d'une recherche linéaire. On dit que la **complexité est logarithmique**.

Avant l'entrée dans la boucle, g a pour valeur 0 et d a pour valeur $\text{len}(\text{liste})$. Si $x < \text{liste}[0]$, ou si $x > \text{liste}[d-1]$, alors x n'est pas dans la liste et la recherche n'aboutira pas.

Il est possible de vérifier avant de commencer la recherche que le nombre cherché est bien compris entre les deux bornes de la liste. On utilise pour cela une assertion qui arrête le programme si la condition à vérifier n'est pas remplie. La syntaxe est `assert liste[g] < x < liste[d-1]`, à placer avant la boucle.

3 CORRECTION

L'algorithme se termine, mais fait-il bien ce que l'on veut ?

Nous ajoutons un élément en fin de liste, strictement supérieur au dernier élément, par exemple égal à $\text{liste}[d-1]$. Nous allons démontrer que la propriété $\text{liste}[g] \leq x < \text{liste}[d]$ qui est vraie avant l'entrée dans la boucle reste encore vraie après chaque passage dans la boucle. Cette propriété est appelée un **invariant de la boucle**. La propriété est vraie avant l'entrée dans la boucle d'après ce qui précède.

Supposons qu'elle soit vraie avant un passage dans la boucle : $\text{liste}[g] \leq x < \text{liste}[d]$.

D'après le choix de m , $\text{liste}[g] \leq \text{liste}[m] < \text{liste}[d]$ puisque la liste est triée.

Donc, si $x < \text{liste}[m]$ on obtient $\text{liste}[g] \leq x < \text{liste}[m]$. Dans ce cas la nouvelle valeur de d est k , et donc $\text{liste}[g] \leq x < \text{liste}[d]$ après le passage dans la boucle.

Sinon, $\text{liste}[m] \leq x$, et on obtient $\text{liste}[m] \leq x < \text{liste}[d]$. Dans ce cas la nouvelle valeur de g est k , et donc $\text{liste}[g] \leq x < \text{liste}[d]$ après le passage dans la boucle.

Il reste à examiner l'état des variables après la sortie de la boucle. Lorsque le dernier passage dans la boucle est effectué, $d - g \geq 2$. Donc $g < m < d$. Si d prend la valeur m ou si g prend la valeur m , alors $d - g > 0$. Mais s'il n'y a plus de passage dans la boucle, cela signifie que $d - g \leq 1$. La seule possibilité est donc $d - g = 1$.

Or $\text{liste}[g] \leq x < \text{liste}[d]$, donc $\text{liste}[g] \leq x < \text{liste}[g+1]$. En conclusion, nous n'avons que deux possibilités : soit x est la valeur de $\text{liste}[g]$, soit x n'est pas dans la liste.

4 LÉGÈRE MODIFICATION

Une nouvelle version, qui fait deux tests par boucle, mais qui teste la présence à chaque étape :

```

1 def dichotomique(x, liste):
2     g, d = 0, len(liste) - 1
3     while g <= d:
4         m = (g + d) // 2
5         if liste[m] == x:
6             return m
7         if liste[m] < x:
8             g = m + 1
9         else:
10            d = m - 1
11    return (None)

```

La première est en général 15 à 20% plus rapide.

Sauf si on tombe rapidement sur l'élément...

II EXPONENTIATION RAPIDE

1 PUISSANCE NAÏVE

Il s'agit d'écrire une fonction puissance qui prend en paramètres un flottant x non nul et un entier naturel n et qui renvoie x^n .

Dans une version naïve, on calcule successivement x^2, x^3 , etc, en se basant sur la définition mathématique : pour $n \geq 0, x^n = 1 \times x \times \dots \times x$ où n est le nombre de facteur égaux à x .

```
1 def puissance1(x,n):
2     p = 1
3     for i in range(n):
4         p = p*x
5     return p
```

La fonction `puissance1` effectue n produits et $n + 1$ affectations. La complexité est donc linéaire en fonction de n .

Dans le calcul du coût, un autre facteur ne doit pas être négligé, la taille de x . En Python il n'y a pas de limite de taille pour les entiers. Donc si x est de type `int`, le temps de calcul peut être nettement allongé selon la taille de x .

2 MÉTHODE DICHOTOMIQUE

Pour tout n , si n est pair, $x^n = (x^2)^{n/2}$, sinon $x^n = x(x^2)^{n/2}$, où $n/2$ désigne le quotient entier de n par 2. Nous en déduisons l'algorithme suivant :

- p vaut 1
- Tant que $n/2$ n'est pas nul :
 - ◊ si n est impair nous remplaçons p par px
 - ◊ nous remplaçons x par x^2 et n par $n/2$.

Le principe d'un algorithme dichotomique est de diviser un problème en deux sous-problèmes.

```
1 def puissance2(x,n):
2     p = 1
3     while n != 0:
4         if n%2 == 1:
5             p = p*x
6             x = x*x
7             n = n//2
8     return p
```

3 COMPLEXITÉ LOGARITHMIQUE

Considérons l'écriture binaire de n (des précisions au deuxième semestre).

$$n = n_0 + n_1 \times 2 + n_2 \times 2^2 + \dots + n_k \times 2^k = \sum_{i=0}^k n_i 2^i$$

avec pour tout i, n_i égal à 0 ou à 1.

Nous pouvons alors écrire : $x^n = x^{\sum_{i=0}^k n_i 2^i} = x^{n_0} (x^2)^{n_1} (x^4)^{n_2} \dots (x^{2^k})^{n_k}$. Dans le cas le moins favorable, nous pouvons considérer que nous avons de l'ordre de $4k$ opérations à effectuer, où k est le nombre de chiffres dans l'écriture binaire de n , soit environ $\log_2(n)$ opérations (k multiplications, k élévations au carré, et k calculs de quotient et de reste). Le logarithme en base b d'un nombre entier n est de l'ordre du nombre de chiffres dans l'écriture en base b de n .

Si $c_{10}(n)$ est le nombre de chiffres dans l'écriture décimale de n , alors nous pouvons en déduire que $c_{10}(n) - 1 \leq \log_{10}(n) < c_{10}(n)$, soit $\log_{10}(n) < c_{10}(n) \leq \log_{10}(n) + 1$.